

ANALYSE DU CODE - A7.2

Projet Jeu d'Echecs - INFO7 L1 Info 2025-2026

A7.2.a : IDENTIFICATION DES 2 FONCTIONNALITES CRITIQUES

FONCTIONNALITE 1 : choose_movement_computer (game.cpp)

Structures :

- Plateau P (tableau de 64 cases de type Case)
- Masque M (tableau de 64 entiers)
- game G (structure globale du jeu)

Fonctions :

- choose_movement_computer() : fonction principale de sélection du coup
- test_run() : vérifie si un mouvement est valide
- highlights_possible_moves() : calcule le masque des mouvements possibles
- king_in_check() : vérifie si le roi est en échec après le coup
- test_coup() : simule un coup sur un plateau temporaire
- move_piece() : déplace une pièce sur le plateau

Cette fonction est la fonction, principale qui définit les mouvements de l'adversaire (ordinateur). Elle doit garantir que chaque coup joué par l'ordinateur est légal (pièce de la bonne couleur, destination valide, roi non mis en échec par son propre mouvement).

FONCTIONNALITE 2 : backtrack_historique (historique.cpp)

Structures :

- historique* (liste chaînée : coord_depart, coord_arrivee, piece_deplacee, piece_prise, suivant)
- Plateau P (tableau de 64 cases)
- game G (structure globale)

Fonctions :

- backtrack_historique() : fonction principale de retour arrière
- move_piece() : utilisée pour annuler le déplacement
- print_board() : affichage du plateau après retour arrière

Elle permet de revenir en arrière dans la partie. Elle doit reconstruire exactement l'état du plateau avant le coup annulé, y compris remettre en place les pièces capturées.

A7.2.b : ANALYSE DES DEUX FONCTIONNALITES

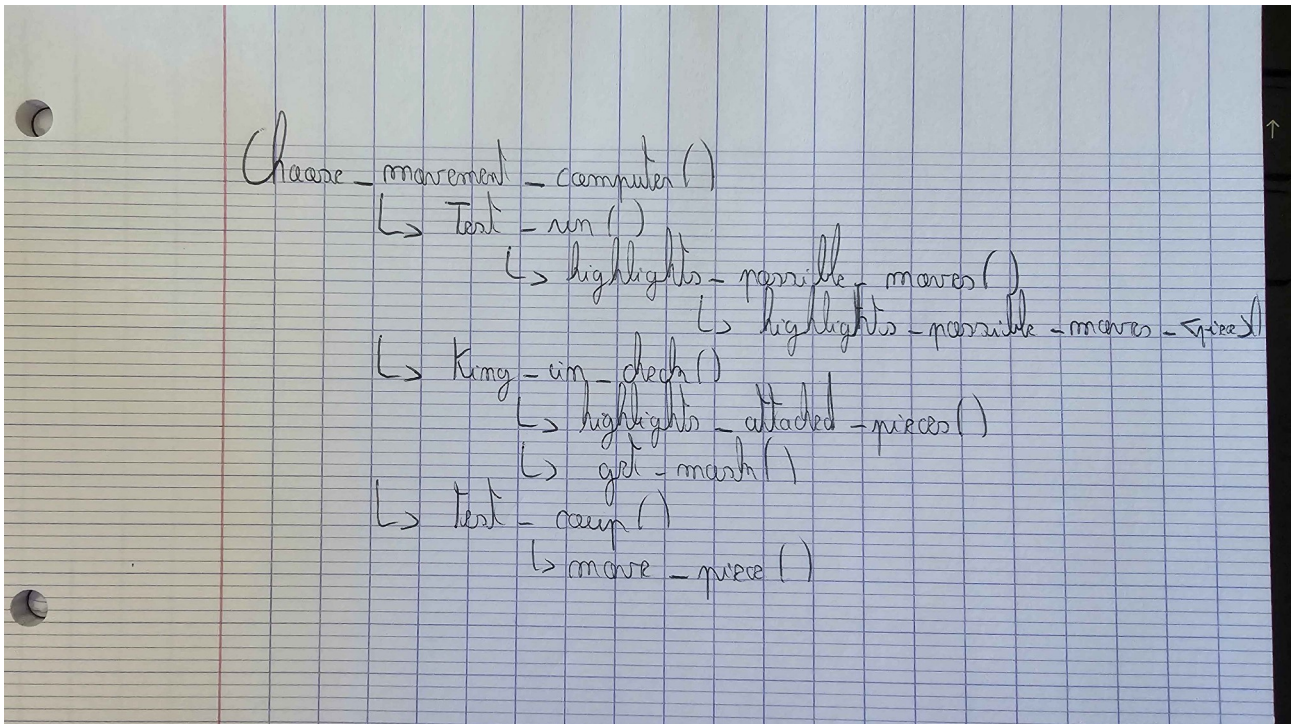
ANALYSE 1 : choose_movement_computer

La fonction tire aléatoirement une case de départ et une case destination (entre 0 et 63), puis vérifie 4 conditions dans le while :

1. La case de départ n'est pas vide
2. La pièce appartient à l'ordinateur (couleur correcte)
3. Le mouvement est valide selon les règles (test_run)
4. Le roi de l'ordinateur n'est pas en échec après ce coup (king_in_check)

La boucle se termine dès qu'un coup valide est trouvé.

Schéma d'appel :



Complexité temporelle

Dans le pire cas, chaque itération de la boucle `while` effectue :

- `test_run` : $O(64)$ pour parcourir le masque
- `king_in_check` : $O(64)$ pour parcourir les pièces adverses + $O(64)$ pour chercher le roi dans le masque $\rightarrow O(64)$
- `test_coup` : $O(1)$

Soit $O(64)$ par itération avec nbre aléatoire

avec 20 coups légaux disponibles en moyenne, et $64 \times 64 = 4096$

paires possibles, la probabilité de tomber sur un coup valide est d'environ 0.5% au premier tirage. Le nombre moyen d'itérations est donc d'environ 200, soit environ $200 \times 64 = 12\,800$ opérations en moyenne par tour.

Complexité spatiale

$O(1)$: uniquement 2 entiers (indices) + 1 plateau temporaire de 64 cases.

Alternatives

Une alternative meilleure serait de :

1. Lister d'abord toutes les pièces jouables dans un tableau ($O(64)$)
2. Pour chaque pièce, lister ses coups légaux avec le masque ($O(64)$)
3. Choisir aléatoirement parmi les coups légaux trouvés
ça ferait $O(64^2)$ dans le pire cas,

ANALYSE 2 : backtrack_historique

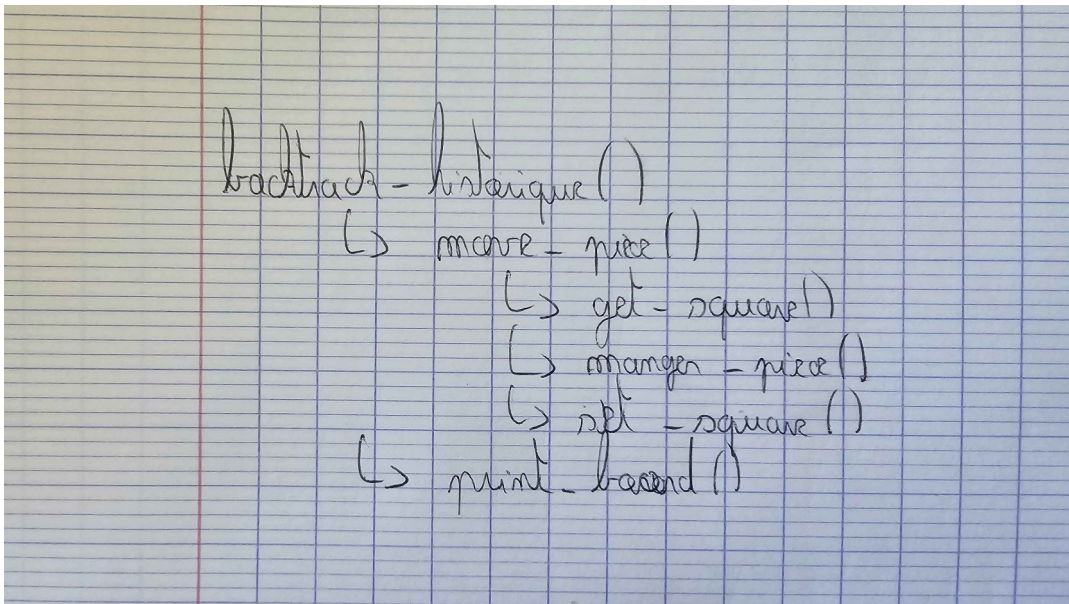
La fonction prend en paramètre le nombre d'étapes à reculer.

Pour chaque étape, elle recule de 2 coups (1 coup humain + 1 coup ordinateur) en défilant la tête de la liste chaînée historique.

Pour chaque coup annulé :

1. On récupère la tête de l'historique ($\text{temp} = G \rightarrow \text{hist}$)
2. On avance le pointeur hist au suivant ($G \rightarrow \text{hist} = G \rightarrow \text{hist} \rightarrow \text{suivant}$)
3. On inverse le déplacement : move_piece de coord_arrivee vers coord_depart
4. On remet la pièce prise à sa place : $\text{Tab}[\text{coord_arrivee}] = \text{piece_prise}$
5. On remet la pièce déplacée : $\text{Tab}[\text{coord_depart}] = \text{piece_deplacee}$
6. On libère la mémoire de l'entrée historique (delete temp)

Schéma d'appel



Complexité temporelle

Pour un retour de N étapes, et 2 coups par étape :

- Chaque annulation de coup : $O(1)$ (accès direct aux cases par indice)

- print_board final : $O(64)$

Complexité totale : $O(N) + O(64) = O(N + 64) \approx O(N)$
N est petit, donc très rapide.

Alternatives

Une alternative serait de sauvegarder l'intégralité du plateau à chaque coup. Le retour arrière serait immédiat mais la mémoire utilisée serait beaucoup plus importante